Software Documentation

for

Composite-Inventory-Manager App

Team: Commitment Issues

Prepared by: Salvatore DeLuca
Malcolm Bailey
Eulogio Vargas
Dohyeon Yoo

Version: 1.6
Date: 04/27/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Revision History iii
1. Introduction 1
1.1 Overview 1
1.2 Intended Audience and Reading Suggestions 1
1.3 Product Scope 1
1.4 Acronyms and Terminology 3
1.5 References 3
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 6
2.3 User Classes and Characteristics 8
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 8
2.6 Assumptions and Dependencies 8
3. External Interface Requirements 9
3.1 User Interfaces 9
3.2 Hardware Interfaces 9
3.3 Software Interfaces 9
4. System Features 9
4.1 Functional Requirements 10
4.2 Non-functional Requirements 24
4.2.1 Performance Requirements 24
4.2.2 Safety Requirements 25
4.2.3 Security Requirements 25
5. Software Design 25
5.1 High Level Design ..... **Error! Bookmark not defined.**
5.2 Detailed Design 27
5.3 Detail Code 29
Appendix A 30
A.1 Traceability Matrix 30
A.2 Test Cases and Results 33

Revision History

Table 1: Revision History

Name	Date	Reason for Changes	Version
Salvatore DeLuca	04/27/2026	Updated formatting and grammar.	1.6
Salvatore DeLuca Malcolm Bailey, Eulogio Vargas, Dohyeon Yoo,	04/20/2026	Updated sections 4.1, 5.3, A.1, and A.2 to reflect current program status.	1.5
Salvatore DeLuca, Dohyeon Yoo, Eulogio Vargas	04/06/2026	Initial creation of section 1.5. Updated sections 5.2, A.1, and A.2 to reflect current case and testing.	1.4
Salvatore DeLuca, Malcolm Bailey, Dohyeon Yoo	03/23/2026	Updated section 4.1-4.2 to reflect code additions. Updated section 5.1-5.2 to reflect code additions. Addition of section 5.3. Addition of section A.1, and A.2.	1.3
Salvatore DeLuca, Malcolm Bailey, Dohyeon Yoo	03/02/2026	Finalized sections 2.0-2.6, 3.0-3.3, and 5.1. Modified information in sections 4.0, and 4.1. Initial creation of section 5.2.	1.2
Salvatore DeLuca, Malcolm Bailey, Dohyeon Yoo,	02/18/2026	Addition of sections 4.1-4.2, and 5.1, modified information in sections 1-3 to reflect additional sections.	1.1
Salvatore DeLuca, Malcolm Bailey, Eulogio Vargas, Dohyeon Yoo,	02/02/2026	Initial creation of software document and completion of sections 1.1-1.3, 2.1-2.6, and 3.1-3.3.	1.0

1. Introduction

This section introduces the software, defining what the piece of software is and the product's goals. Included in this section is the software Overview, Intended Audience and Reading Suggestions, Product Scope, Acronyms and Terminology, and References

1.1 Overview

The Composite-Inventory-Management App is a standalone program created for the Aviation Maintenance Science (AMS). Composites lab staff and professor Travis Billette document the current inventory of the composites lab. The Composite-Inventory-Management App inputs items into an updatable database and organizes them to be easy to view, understand, and modify. Items will be able to be listed in categories determined by the user and predetermined in the program. Currently, the AMS lab inventory is being tracked on an excel sheet, inventory is hard to manage lab TAs leading to inconsistencies, locations of inventory are difficult to keep track of, and the current method to keep track of inventory is time consuming. Implementation of inventory-management software will greatly reduce error and inconsistencies in inventory management, track general locations of where inventory is physically held, and reduce time taken by lab staff to manage inventory.

1.2 Intended Audience and Reading Suggestions

The Composite-Inventory-Management App is designed for sole use by the composite lab at Embry-Riddle Aeronautical University. Specifically, the lab TA's assist with inventory management and resource availability. Section 2.1 (product perspective), as well as section 2.2 (product functions) will be of great use to the lab TAs. These sections provide in greater depth a look into the Composite-Inventory-Management App, what the software means to accomplish, and what features the app will provide to the composite lab.

1.3 Product Scope

The product is divided into 5 categories: Database, Item Creation and Storage, Item Details and Management, Category Management, and User Login. The scope of each category is listed below.

In-Scope:

1. Database Access and Management
    a. The program will connect to a database to store and retrieve inventory data.
    b. The database information will be saved after each modification to item's entries.
2. Item Creation and Storage
    a. Users will be able to create new inventory items in the database.
    b. Each item will have a Unique Identifier (UID), and its information can be accessed, updated, or deleted.
3. Item Details Management
    a. The program will allow users to add, modify, or delete the following details for each item:
        i. Name
        ii. Quantity
        iii. Date added

        iv. Expiration Date
        v. Storage location
        vi. Category
        vii. Restock status
4. Category Management
    a. Users will be able to create and manage inventory categories.
    b. Items can be assigned to one or more categories.
5. User Login
    a. The program will require the user to enter a password before accessing the program.
    b. A Backup Security PIN maybe be used in the password field to access the program.
6. Database Query
    a. The user will be able to sort through the inventory based on the items attributes:
        i. Location
        ii. Expiration date
        iii. Quantity
        iv. Category

The Out-of-Scope items are features which the software will be developed. The feature and its reason for being Out-of-Scope is listed below.

Out-of-Scope:

1. Internet Access
    a. This program will not be accessible online on a website as it will be pushing the project out of budget and schedule.

The program has room for many improvements to functionality and user connivence. Below is a list of additional features that may be added after the in-scope features are completed.

1. Change Logs
    a. Every change the user makes to the database is recorded in a change log file.
    b. The user can view the change log document.
2. Printing
    a. Users will be able to save a PDF file and print out a full report of all items and details.
    b. Users will be able to save a PDF file and print out selected individual items or categories and their details.
3. Backup
    a. The user will be able to create backup files.
    b. The user may be able to restore the database using a backup file.

1.4 Acronyms and Terminology

Table 2: Table of Acronyms and Terminology

Acronym	Term	Definition
CIM	Composite Inventory Management	Short for Composite Inventory Management the name of the software.
AMS	Aviation Maintenance Safety	The university department focused on aviation maintenance.
TA	Teachers Assistant	A student in charge of assisting the teacher/ professor.
GUI	Graphical User Interface	A type of interface that allows the user to interact with the program using visual elements instead of text.
PIN	Personal Identification Number	Security number used as an authentication for the user's identity.
UID	Unique Identifier	An automatically assigned number to database entries.
SQL	Structured Query Language	The standard language used to create, retrieve, and manipulate data within the database.
PDF	Portable Document Format	A file format used for generating and printing reports of items.
ERD	Entity Relationship Diagram	A visual representation of the different entities within the system and their relations to each other
UML	Unified Modeling Language	A standardized model language for visualizing the design and architecture of the software.
OS	Operating System	The system software that manages the computer hardware and software resources.
USB	Universal Serial Bus	The hardware interface used for the flash drive that hosts the application and database.

1.5 References

N/A

2. Overall Description

This section of the document shall further define the software and its limitations. This section includes Product Perspective, Product Functions, User Classes and Characteristics, Operating Environments, Design and Implementation Constraints, as well as Assumptions and Dependencies.

2.1 Product Perspective

The Composite-Inventory-Management app is a database-driven inventory management system designed to store item information and present it in an organized manner. This program allows the user to create items and details, store them in a database, retrieve them to view, modify them to update any details, and organize them into categories.

Database

The program stores all information in a database. This acts as a storage for all the items and its details. When the program displays a list of items, quantity of each item, and any details of the item, the information of the items is retrieved from the database. When the user creates a new item, modify details, or deletes an item, the actions reflect in the database.

The Software

The software acts as an interface to access the database. The software organizes inventory information in a structured and readable format, making it easier for users to manage items and their associated details. It allows users to interact with stored inventory data without direct access to the database itself. When a user requests to view inventory data, the software retrieves the relevant records from the database and presents them in an organized manner.

Through the software, users can create new inventory items, view existing items, modify item details, delete the item, ond organize items into categories. All user actions are then processed by the program, made into the database, and retrieved back into the program.

Item Management

The program provides functionality for managing inventory items throughout their lifecycle. Users can create new items, each of which is assigned to a unique identifier (UID). This identifier allows the system to locate individual items when retrieving, updating, or deleting records. Users can modify item details such as quantity, date added, expiration date, storage location, category, and restock status to organize and update information.

Category Management

The software allows for organization and filtering of inventory items through categories. Users can create and manage categories to group related items together. Items may be assigned to one or more categories.

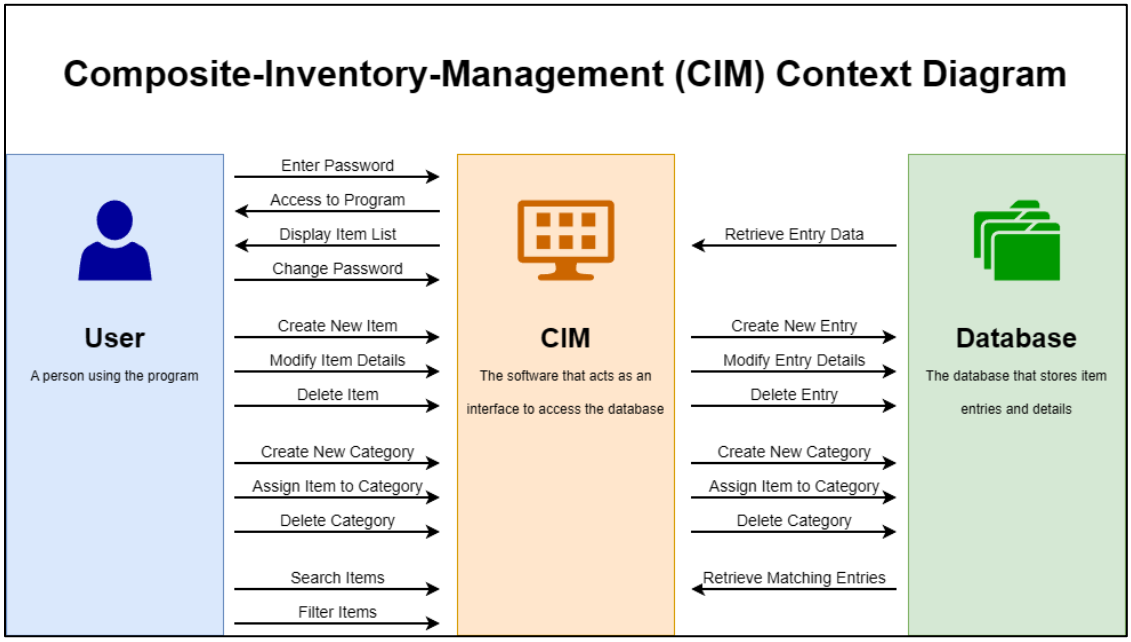
To-Buy-List

The software allows the user to view the To-Buy-List and add/remove items to the To-Buy-List. The To-Buy-List is a separate data table which the user can add or remove items from. The To-Buy-List will show the item, and all its details in addition to the quantity needed to be purchased and the vendor from which the item was purchased from. The user may add or remove vendors from the associated item.

User Login

The software is secured via password authentication to prevent unauthorized access. The user must enter the set password into the password field. The password may be changed in the settings. In case the password is lost, a permanent backup security code will be provided to the software owner. The backup security code may be entered into the password field, and the password must be changed. The Context Diagram depicts the data flow and interactions between the three primary entities in the system: the User, the CIM software, and the Database. On the left, the User (a person using the program, specifically the composites lab TA) sends commands to the CIM software including creating new items, modifying items, deleting items, modifying item details, entering passwords for authentication, and receives outputs such as displayed item lists and program access confirmation.

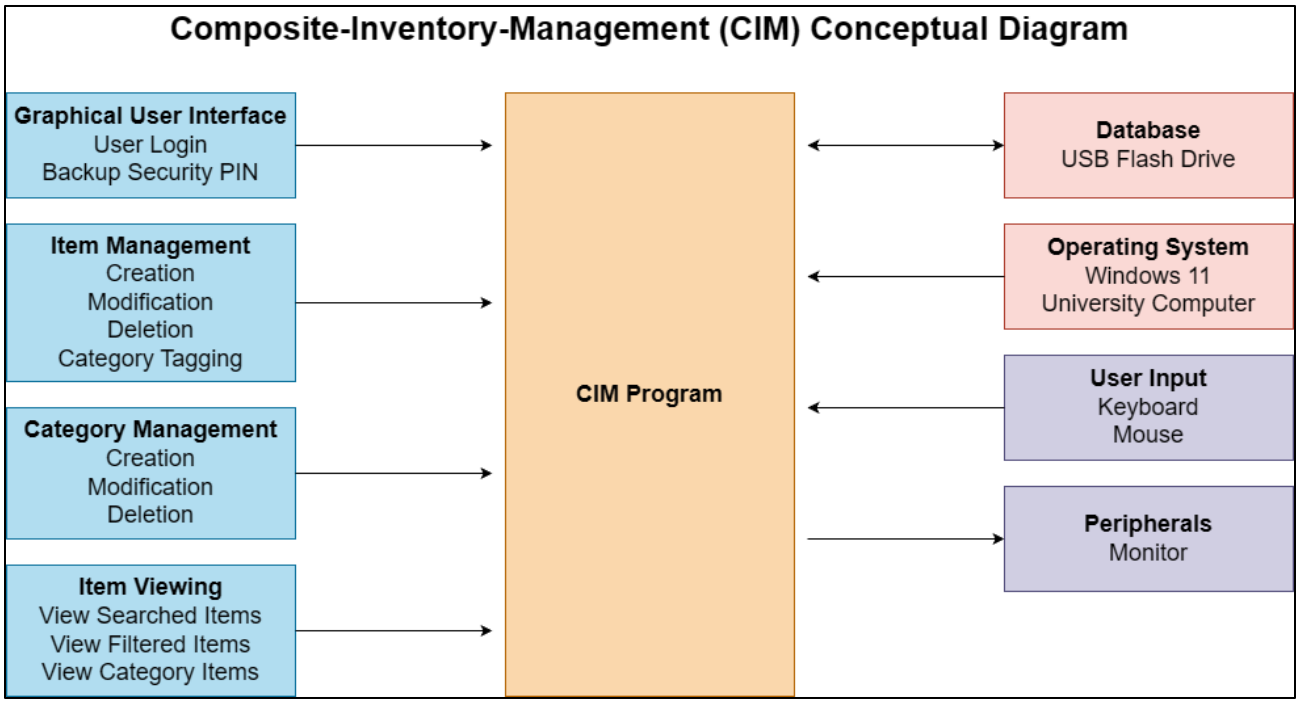
Context Diagram



In the center, the CIM software acts as an interface between the user and the database, processing all user requests and coordinating data operations.

On the right, the Database stores all item entries and details, receiving commands from the CIM software to create new entries, modify entries, delete entries, and modify entry details, while sending retrieved entry data back to the CIM software for display to the user. The arrows indicate the direction of data flow, showing how user actions are processed by the CIM software and translated into database operations, with the results flowing back through the CIM software to the user, illustrating the software's central role in mediating all interactions between the human user and the persistent data storage.

Conceptual Diagram

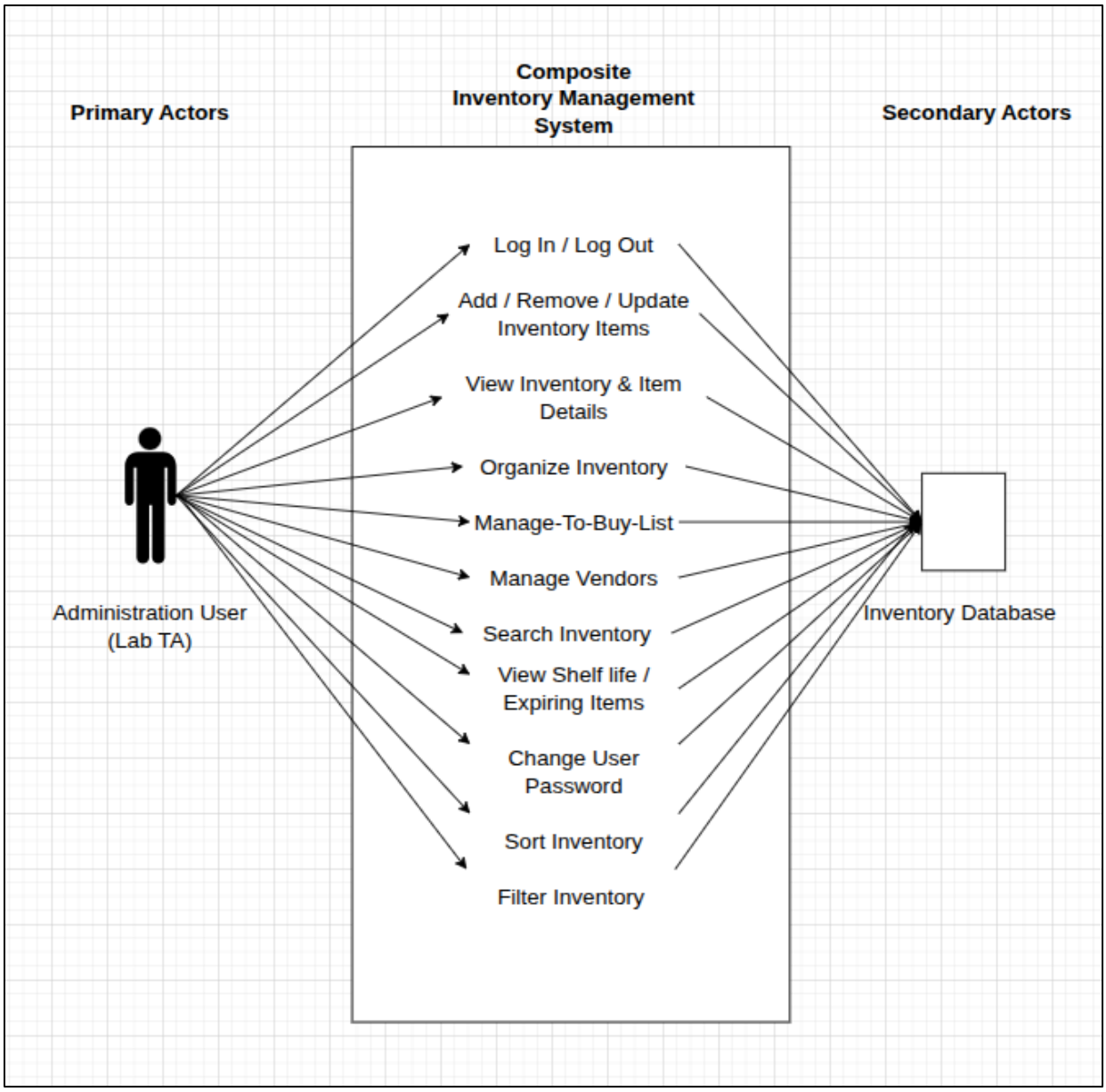


The Conceptual Diagram illustrates the relationships of the components of the system with external entities at the highest level of functionality. The Graphical User Interface components located on the left portion of the figure present the primary areas of functionality: User Login with Backup Security PIN for authenticating users, Item Management for creating, modifying, deleting, and categorizing the inventory items, and Category Management for managing the categories of the inventory groups.

The components of the Graphical User Interface work in conjunction with the main components of the CIM Program. On the right portion of the figure, it is evident that the components of the external entities the CIM Program works with are the Database residing on the USB Flash Drive for persistent data storage needs, the Operating System residing on the university computer with Windows 11 as the installed OS, the User Input components for receiving user input (keyboard and mouse), and the Peripherals for the output of the system (monitor).

2.2 Product Functions

The following diagram is a use case diagram that clearly defines the possible actions and relationships between the primary actors (left), system actions (middle), and secondary actors (right). The Composite-Inventory-Management App has only one primary actor, being the Composite Lab TA, as access to the app is only meant for use by an administrative user. The secondary actors are the inventory database that the app will interact with to fetch records for viewing or updating records.



User Functions

- Log In
- Log Out
- Add Inventory Items
- Remove Inventory Items
- Update Inventory Records
- View Inventory Records and Item Details
- Organize Inventory by groups
- Add items to the –To-Buy-List
- Remove items from the To-Buy-List
- Update item quantity on the To-Buy-List

- Add Vendors to Items on the To-Buy-List
- Remove Vendors from Items on the To-Buy-List
- Search through Inventory Database
- View Shelf Life of Items
- Manage Shelf Life of Items
- Sort Inventory
- Filter Inventory

2.3 User Classes and Characteristics

This section will encompass the types of users for our product along with their respective purposes and characteristics. Our product shall have the following user classes:

Developer

The developer shall have full access to the software and databases to conduct product design and testing.

User

The User shall have the ability to reference the material database via the GUI to adjust material properties.

2.4 Operating Environment

The Composite-Inventory-Management App is designed to be run via a Windows computer currently operating Windows 11 or Windows 10. Cross operating system support (i.e. MacOS, Linux) is not supported. The app will be run on a singular flash drive that will stay in the composites laboratory located inside. The app will work alongside the current organization system within the laboratory.

2.5 Design and Implementation Constraints

- The product shall contain a secure login system that follows modern encryption practices
- The product shall contain a menu driven Graphical User Interface (GUI)
- The product shall contain a database of materials held within the materials laboratory
- The product shall operate on a provided USB-A flash drive

2.6 Assumptions and Dependencies

To successfully run the app on the user's computer, it is assumed that the user has access to the flash drive containing the Composite-Inventory-Management app, as well as a USB-A port on their laptop or desktop PC. It is required that the user has Windows 10 or Windows 11 installed as their current operating system. No internet access is required for the app to operate. It is assumed that the flash drive will be kept in a dry environment away from moisture to ensure proper functions of the hardware.

3. External Interface Requirements

This section will list specific qualities of the interfaces of the Composite-Inventory-Management App. System interfaces are defined as User Interfaces, Hardware Interfaces, and Software Interfaces.

3.1 User Interfaces

UI-1: The system shall provide a Graphical User Interface (GUI) that allows the admin to interact with the program visually.

UI-2: The system shall incorporate a login interface requiring a password and include functionality to reset the password.

UI-3: The system shall allow the administrator to add, remove, update, and view inventory items.

UI-4: The system shall allow the admin to search for inventory items by name, category, or filter criteria using a search input field.

UI-5: The system shall allow the administrator to associate vendors with inventory items and add selected items to a to-buy list.

UI-6: The system shall display inventory records in a structured and readable tabular format within the interface.

3.2 Hardware Interfaces

HI.1: The system shall be executable through a USB-A flash drive when the flash drive is connected to a Windows 10 or 11 based computer.

HI.2: The system shall accept user input through a keyboard and mouse and display output on a standard computer monitor when operated on supported hardware.

3.3 Software Interfaces

SI-1: The system shall operate on the Microsoft Windows 10 or 11 operating system.

SI-2: The system shall interface with a local SQLite database to store and retrieve inventory and user data.

SI-3: The system shall be executed as a standalone application without requiring network-based services or external cloud platforms.

4. System Features

This section will describe system features; it will describe what the App is capable of and what the software can be used for. This section consists of Functional Requirements and Non-

functional Requirements. App use cases and performance, safety, and security requirements will also be addressed in this section.

4.1 Functional Requirements

Use Case ID: UC1
Use Case Name: Add a new item to the database
Goal: Create a new inventory item and store it in the database
Actors: User, Database
Pre:

- The user is authenticated and logged into the system
- The database connection is active

Post:

- A new inventory item is created and stored in the database with a unique ID

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System displays a list of all lab tools and materials
5	User selects “Create Item”	6	System displays item creation form
7	User enters item details	8	System validates entered data
9	User submits item	10	System generates a unique item ID
11	System stores item and details in the database	12	System confirms item creation

Exceptions (Alternative Scenarios of Failure):

- The user enters the wrong password
- User enters incomplete or invalid item data
- Database connection failure
- User cancels item creation

Functional Requirements

- Req. 1: The system shall provide an option to create a new inventory item
- Req. 2: The system shall allow the user to enter item details
- Req. 3: The system shall validate item data before submission
- Req. 4: The system shall generate a unique identifier for each item
- Req. 5: The system shall store the item and its details in the database
- Req. 6: The system shall confirm successful item creation to the user

Use Case ID: UC2
Use Case Name: Modify Inventory Item
Goal: Modify the quantity of an existing inventory item
Actors: User, Database
Pre:

- The user is authenticated and logged into the system

- The inventory item exists in the database
- The database connection is active

Post:
- The selected inventory item is updated in the database

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects “Modify Item”	6	System displays item list
7	User selects an item	8	System retrieves item quantity
9	User modifies item quantity	10	System validates updated data
11	System updates item in the database	12	System confirms item modification

Exceptions (Alternative Scenarios of Failure):
- User selects an invalid or non-existent item
- Invalid data entered for item details
- Database update failure

Functional Requirements
- Req. 7: The system shall provide a list of existing inventory items
- Req. 8: The system shall allow the user to select an inventory item
- Req. 9: The system shall display current item quantity
- Req. 10: The system shall allow modification of item quantity
- Req. 11: The system shall validate modified item data
- Req. 12: The system shall update the item in the database
- Req. 13: The system shall confirm successful item modification

Use Case ID: UC3
Use Case Name: Delete Inventory Item
Goal: Remove an inventory item from the database
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The inventory item exists in the database
- The database connection is active

Post:
- The inventory item is permanently removed from the database

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in

3	The user enters proper login information	4	System presents the main menu
5	User selects “Delete Item”	6	System displays item list
7	User selects an item	8	System requests deletion confirmation
9	User confirms deletion	10	System removes item from the database
11	System confirms item deletion		

Exceptions (Alternative Scenarios of Failure):
- User cancels deletion
- User selects an invalid or non-existent item
- Database deletion failure

Functional Requirements
- Req. 7: The system shall provide a list of existing inventory items
- Req. 8: The system shall allow the user to select an inventory item
- Req. 14: The system shall confirm the selected item with the user
- Req. 15: The system shall remove the item and its details from the database
- Req. 16: The system shall confirm the item deletion to the user

Use Case ID: UC4
Use Case Name: View Inventory List
Goal: Display all inventory items
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active

Post:
- Inventory items displayed to user

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	Items are displayed in rows.

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database

Functional Requirements
- Req. 8: The system shall allow the user to select an inventory item
- Req 17: The system shall retrieve item details from the database.
- Req 18: The system shall display inventory items in a tabular spreadsheet format.

Use Case ID: UC5
Use Case Name: View Item Details
Goal: Display full details of a selected inventory item
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists in database

Post:
- Inventory items are displayed to user

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	Item details are displayed in columns.

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database

Functional Requirements
- Req. 8: The system shall allow the user to select an inventory item
- Req 17: The system shall retrieve item details from the database.
- Req 19: The system shall display item details such as quantity, category, and expiration date.

Use Case ID: UC6
Use Case Name: Search Inventory Item
Goal: Locate inventory items that match user-provided search criteria
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Inventory items exist in the system

Post:
- Matching inventory items are displayed to the user
-

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu

5	User selects “Search Item”	6	System displays search input field
7	User enters search criteria	8	System validates search input
9	System queries the database	10	System retrieves matching items
11	System displays search results		

Exceptions (Alternative Scenarios of Failure):
- User enters invalid search input
- No matching items found
- Database connection failure

Functional Requirements
- Req 20: The system shall display a search input option
- Req 21: The system shall validate the search criteria when search input is submitted
- Req 22: The system shall retrieve inventory items matching the search criteria
- Req 23: The system shall display matching inventory items
- Req 24: The system shall notify the user when no matching inventory items are found in response to a search request.

Use Case ID: UC7
Use Case Name: Sort Inventory Item
Goal: Arrange inventory items according to selected sorting criteria
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Inventory items exist in the system

Post:
- Inventory items are displayed in the selected order

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User views the inventory list	6	System displays inventory items in tabular format with item detail category headers
7	User clicks on a column header (e.g., Quantity, Expiration Date, Category)	8	System detects selected column header
9	System sorts inventory items in ascending order based on selected column	10	System updates and displays the sorted inventory list
11	User clicks the same column header again	12	System sorts inventory items in descending order
13	System updates and displays the sorted inventory list		

Exceptions (Alternative Scenarios of Failure):
- Database connection failure

- No inventory items in the database

Functional Requirements
- Req 25: The system shall display a sort item option
- Req 26: The system shall retrieve inventory items matching the sort criteria
- Req 27: The system shall display matching inventory items
- Req 28: The system shall notify the user when no matching inventory items are found in response to a sort request.

Use Case ID: UC8
Use Case Name: Filter Inventory Item
Goal: Display inventory items based on a selected attribute
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Inventory items exist in the system

Post:
- The inventory items are displayed based on the selected filtering attribute

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User views the inventory list	6	System displays inventory items in tabular format with item detail category headers
7	User selects the filter drop down menu	8	System displays a list of filter options
9	User selects a filter option	10	The System only displays the items based on the filter option

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database

Functional Requirements
- Req 29: The system shall display a filter option
- Req 30: The system shall have a set number of options to filter by
- Req 31: The system shall have a graphical method for selecting a filter option
- Req 32: The system shall only display items that match the filter setting
- Req 33: The system shall notify the user when no matching inventory items are found in response to a filter request.

Use Case ID: UC9
Use Case Name: Create Category
Goal: Create a new inventory category
Actors: User, Database
Pre:
- The user is authenticated and logged into the system

- The database connection is active

Post:
- A new category is stored in the database

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects "Create Category"	6	System displays Category Creation screen
7	User enters category name	8	System validates the category name
9	System stores the category in the database	10	System confirms category creation

Exceptions (Alternative Scenarios of Failure):
- User cancels creation
- User selects an invalid or duplicate category name
- Database connection failure

Functional Requirements
- Req. 34: The system shall display an option to create a category in the item management menu
- Req. 35: The system shall display a category creation page with a textbox, "submit" button, and a "cancel" button
- Req. 36: The system shall validate the category name in response to the user submitting a name
- Req. 37: The system shall store the new category in the database in response to successful validation
- Req. 38: The system shall confirm successful category creation to the user in response to storing the category

Use Case ID: UC10
Use Case Name: Delete Category
Goal: Delete an existing inventory category
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- A category exists in the database

Post:
- Specified category removed from database

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in

3	The user enters proper login information	4	System presents the main menu
5	User selects “Delete Category”	6	System displays Category deletion screen
7	User enters category name	8	System validates entered input
9	System removes category from database		

Exceptions (Alternative Scenarios of Failure):
- User cancels deletion
- User selects an invalid category name
- Database connection failure

Functional Requirements
- Req. 39: The system shall display an option to delete a category in the item management menu
- Req. 40: The system shall display a category deletion page with a textbox, “submit” button, and a “cancel” button
- Req. 41: The system shall remove the specified category from the database in response to successful validation

Use Case ID: UC11
Use Case Name: Assign Item to Category
Goal: Assign one or more categories to an inventory item
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The inventory item exists in the database
- The category exists in the database
- The database connection is active

Post:
- The selected item is associated with the selected category or categories

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects	6	System displays
7	User enters category name	8	System
9	System	10	System confirms category

Exceptions (Alternative Scenarios of Failure):
- User cancels creation
- User selects an invalid or duplicate category name
- Database connection failure

Functional Requirements
- Req. 42: The system shall display an option to assign a category in item details

- Req. 43: The system shall display a category page showing all created categories
- Req. 44: The system shall assign the specified category to an item in the database

Use Case ID: UC12
Use Case Name: Unassign Item from Category
Goal: Remove category association from item
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists in database
- Item assigned to category

Post:
- Item no longer linked to category

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	User selects item
9	System displays modification menu	10	User selects remove category
11	System displays associated categories	12	User selects the associated categories
13	User confirms	14	System removes association
15	System updates database		

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database
- Item not assigned to category

Functional Requirements
- Req. 45: The system shall display categories assigned to an item.
- Req. 46: The system shall allow the user to remove a category association.
- Req. 47: The system shall update the database after category removal.

Use Case ID: UC13
Use Case Name: View Items by Category
Goal: Display items belonging to selected category
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Category exists

Post:
- Items in category displayed

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	User selects the filter drop down menu
9	System displays a list of filter options	10	User selects a category
11	System retrieves category items	12	System displays filtered inventory table

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- Category does not exist

Functional Requirements
- Req 48: The system shall allow the user to select a category.
- Req 49: The system shall retrieve items associated with that category.
- Req 50: The system shall display those items in the inventory table.

Use Case ID: UC14
Use Case Name: Manage vendor to Item
Goal: Control vendor information associated with an inventory item
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists

Post:
- Vendor information modified

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
		7	System displays items in tabular spreadsheet format.
8	User selects item	9	System displays menu options
10	User selects "Edit Vendor”	11	System displays a textbox to edit vendor
12	User enters or deletes vendor name		
13	User confirms	14	System stores vendor association to database

Exceptions (Alternative Scenarios of Failure):
- Database connection failure

- No inventory items in the database

Functional Requirements
- Req 51: The system shall allow the user to enter vendor information.
- Req 52: The system shall store vendor information associated with the item.

Use Case ID: UC15
Use Case Name: View Vendors for Item
Goal: Display vendors associated with an item in the database
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists
- Item has vendor entries

Post:
- Vendor list displayed

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	Associated vendor(s) are displayed in a column

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database
- Items are not associated to any vendors

Functional Requirements
- Req 53: The system shall retrieve vendor records for a selected item.
- Req 54: The system shall display associated vendors.

Use Case ID: UC16
Use Case Name: Add Item to To-Buy List
Goal: Mark item as needing purchase
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists

Post:
- Item added to To-Buy list

Main Success Scenario:

Step	Actor Action	Step	System Reaction

1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View Inventory'	6	System retrieves inventory records
7	System displays items in tabular spreadsheet format.	8	User selects item
9	System displays menu options	10	User selects "Add to To-Buy”
11	System adds the entry to database		

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database
- Item is already in the To-Buy list

Functional Requirements
- Req 55: The system shall provide an option to add an item to the To-Buy list.
- Req 56: The system shall store the To-Buy list entry in the database.

Use Case ID: UC17
Use Case Name: Remove Item from To-Buy List
Goal: Remove item from To-Buy list
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists
- Item exists in To-Buy list

Post:
- Item removed from To-Buy list

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View To-Buy List'	6	System retrieves To-Buy list records
7	System displays To-Buy items in tabular spreadsheet format.	8	User selects item
9	System displays menu options	10	User selects "Remove from To-Buy”
11	System removes To-Buy entry from the database	12	System displays updated To-Buy list

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database
- No Item exist in the To-Buy list

Functional Requirements
- Req 57: The system shall allow the user to remove items from the To-Buy list.

- Req 58: The system shall update the database after removal.
- Req 59: The system shall display the updated To-Buy list after removal.

Use Case ID: UC18
Use Case Name: View To-Buy List
Goal: Display items marked for purchase
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active

Post:
- To-buy list displayed

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View To-Buy List'	6	System retrieves To-Buy list records
7	System displays To-Buy items in tabular spreadsheet format.		

Exceptions (Alternative Scenarios of Failure):
- Database connection failure

Functional Requirements
- Req 60: The system shall retrieve To-Buy list records from the database.
- Req 61: The system shall display the To-Buy list to the user.

Use Case ID: UC19
Use Case Name: Generate To-Buy List Report
Goal: Create a text file containing all items in the To-Buy List with their details
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active
- Item exists
- Item exists in To-Buy list

Post:
- A text file containing all To-Buy List items and their details is created

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects 'View To-Buy List'	6	System retrieves To-Buy list records

7	System displays To-Buy items in tabular spreadsheet format.	8	User selects "Generate To-Buy List Report"
9	System retrieves all items in the To-Buy List from the database	10	System gathers item details for each To-Buy List entry
11	System creates a text file containing the item information	12	System confirms successful creation of the text file

Exceptions (Alternative Scenarios of Failure):
- Database connection failure
- No inventory items in the database
- No Item exist in the To-Buy list
- File creation failure
- User cancels report generation

Functional Requirements
- Req 62: The system shall allow the user to select an option to generate a To-Buy List report.
- Req 63: The system shall retrieve all items contained in the To-Buy List from the database.
- Req 64: The system shall retrieve item details associated with each To-Buy List entry.
- Req 65: The system shall create a text file containing all To-Buy List items and their details.
- Req 66: The system shall confirm successful creation of the text file to the user.

Use Case ID: UC20
Use Case Name: User Login
Goal: Authenticate a user to access the inventory system
Actors: User, Program
Pre:
- The system is powered on
- The user has a valid password or backup security code

Post:
- User is successfully authenticated and granted access to the system

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	User enters password	4	System validates the password
5	System grants access to the main menu		

Exceptions (Alternative Scenarios of Failure):
- User enters an incorrect password
- User enters an invalid backup security code
- Authentication system failure

Functional Requirements
- Req. 67: The system shall present a login screen to the user
- Req. 68: The system shall allow the user to enter a password
- Req. 69: The system shall validate the entered password

- Req. 70: The system shall grant access upon successful authentication
- Req. 71: The system shall deny access upon authentication failure

Use Case ID: UC21
Use Case Name: Change Password
Goal: Allow a user to change their password
Actors: User, Database
Pre:
- The user is authenticated and logged into the system
- The database connection is active

Post:
- The user's password is updated

Main Success Scenario:

Step	Actor Action	Step	System Reaction
1	User loads the system	2	System prompts the user to log in
3	The user enters proper login information	4	System presents the main menu
5	User selects "Settings"	6	System displays Settings screen
7	User selects "Change Password"	8	System prompts the user to enter current password or back up PIN
9	User enters current password or back up PIN	10	System prompts the user to enter a new password
11	User enters a new password	12	System modifies the password in the database
13	System confirms the password change to the user		

Exceptions (Alternative Scenarios of Failure):
- User cancels password change
- User fails to enter the old password or back up PIN correctly
- User enters the old password as the new password

Functional Requirements
- Req 72: The system shall allow an authenticated user to request a password change
- Req 73: The system shall verify the current password before accepting a new password
- Req 74: The system shall confirm password changes to the user

4.2 Non-functional Requirements

This section will quantify what non-functional requirements the Composite-Inventory-Management App will have, such as performance, security, or safety requirements.

4.2.1 Performance Requirements

PE.1 The system shall save database records upon shutting down the application

PE.2 The system shall use minimal system resources and not affect normal host operations

PE.3 The System shall maintain stability without unexpectedly failing or crashing

PE.4 The system shall launch within a reasonable time (10 seconds) on a university computer

PE.5 The system shall be executed natively from Windows 10/11 without requiring installation of third-party dependencies or software

PE.6 The system shall close within a reasonable time after the user requests to end the program.

4.2.2 Safety Requirements

SA.1 The system shall comply with all Embry-Riddle university policies regarding acceptable software use, data storage, and execution.

SA.2 The system shall not modify system files or data entries on the host computer

SA.3 The system shall store all application data on the USB flash drive and not leave residual data on the host computer after exiting the application

4.2.3 Security Requirements

SE.1 The system shall restrict application usage to only authenticated and authorized users

SA.2 The system shall follow modern industry acceptable cryptographic techniques to securely store and verify user credentials for authentication

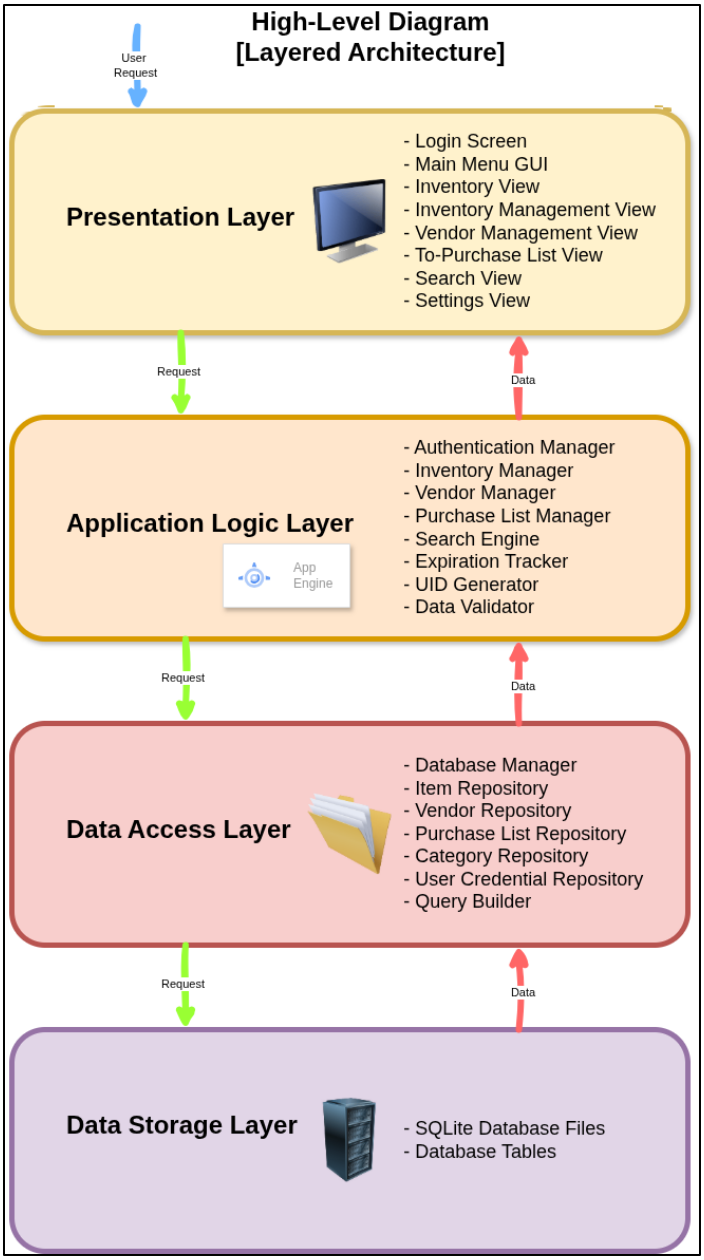
SE.3 The system shall sanitize and validate user input data before processing or storing inside of the database to prevent injection attacks and data corruption

SE.4 The system shall utilize password hashing to securely handle credential storage, and authentication.

5. Software Design

This section will provide insights into the software design of the Composite-Inventory-Management App. The software design can be broken down into High Level Design, Detailed Design, and Detailed code. This provides a top-down overview of software design.

5.1 High Level Design



Presentation Layer

The Presentation Layer contains all graphical interface components that the lab TA interacts with directly. This includes the login screen for authentication, the main dashboard for navigation, inventory view panels displaying items with quantities and expiration dates, item management forms for adding and editing entries, vendor management panels for associating suppliers with items, the to-purchase list for tracking items needing restock, a search bar for querying inventory, and settings dialogs for password management.

Business Logic Layer

The Business Logic Layer processes user requests and enforces application rules. Key components include the authentication controller for validating credentials, the inventory manager for item operations and expiration tracking, the vendor manager for maintaining item-supplier relationships, the purchase list controller for managing restock items, the search engine for filtering inventory, the UID generator for creating unique identifiers, and the data validator for sanitizing input. This layer requires no session management since the application supports only a single user.

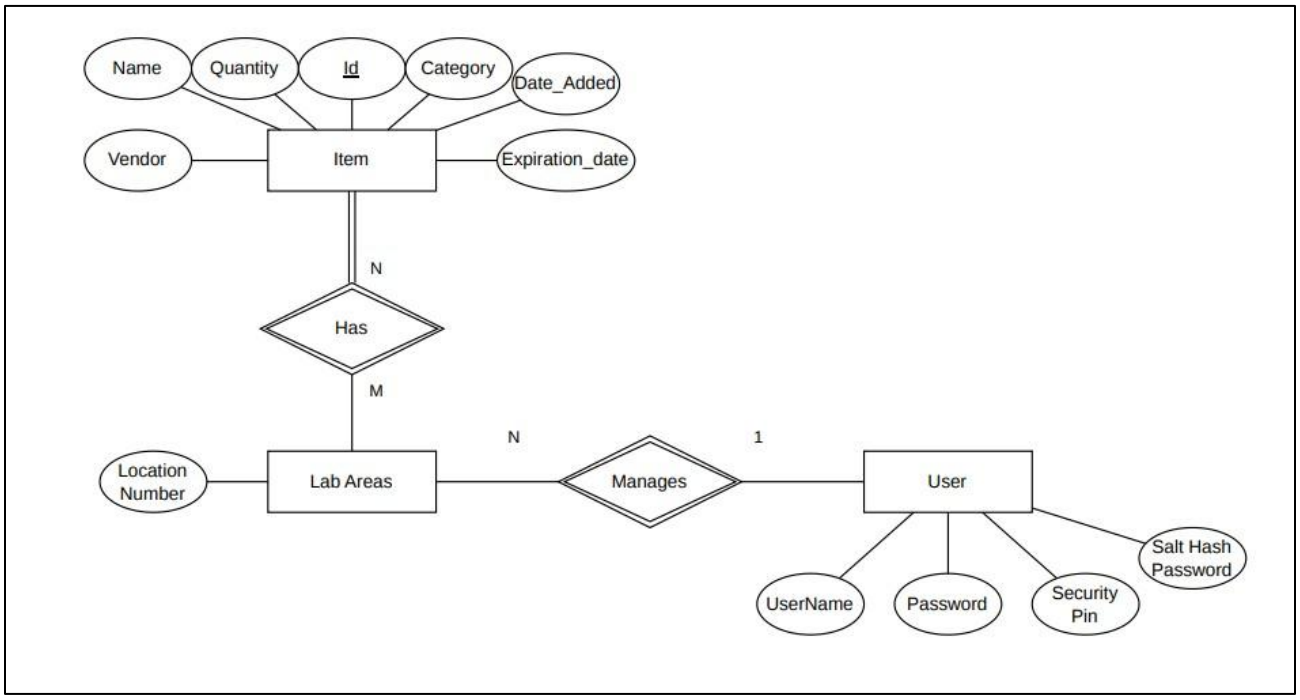
Data Access Layer

The Data Access Layer abstracts database operations through specialized repositories and connection management. The database connection manager maintains the SQLite connection using relative paths for USB portability, while dedicated repositories handle items, vendors, purchase lists, categories, and user credentials. The query builder constructs parameterized SQL statements to prevent injection attacks. This layer ensures the application works regardless of which drive letter is assigned to the USB flash drive.

Database Layer

The Database Layer consists of a SQLite database file stored on the USB flash drive containing all persistent data. Tables include Items (UID, name, quantity, location, expiration date, category, restock status), Vendors (supplier information), Item Vendors (many-to-many relationships), Categories (organizational groupings), Purchase List (items needing restock with quantities), and User (hashed password and backup PIN). SQLite was chosen for its portability, zero-configuration requirements, and ability to operate entirely from the USB drive without installation.

5.2 Detailed Design



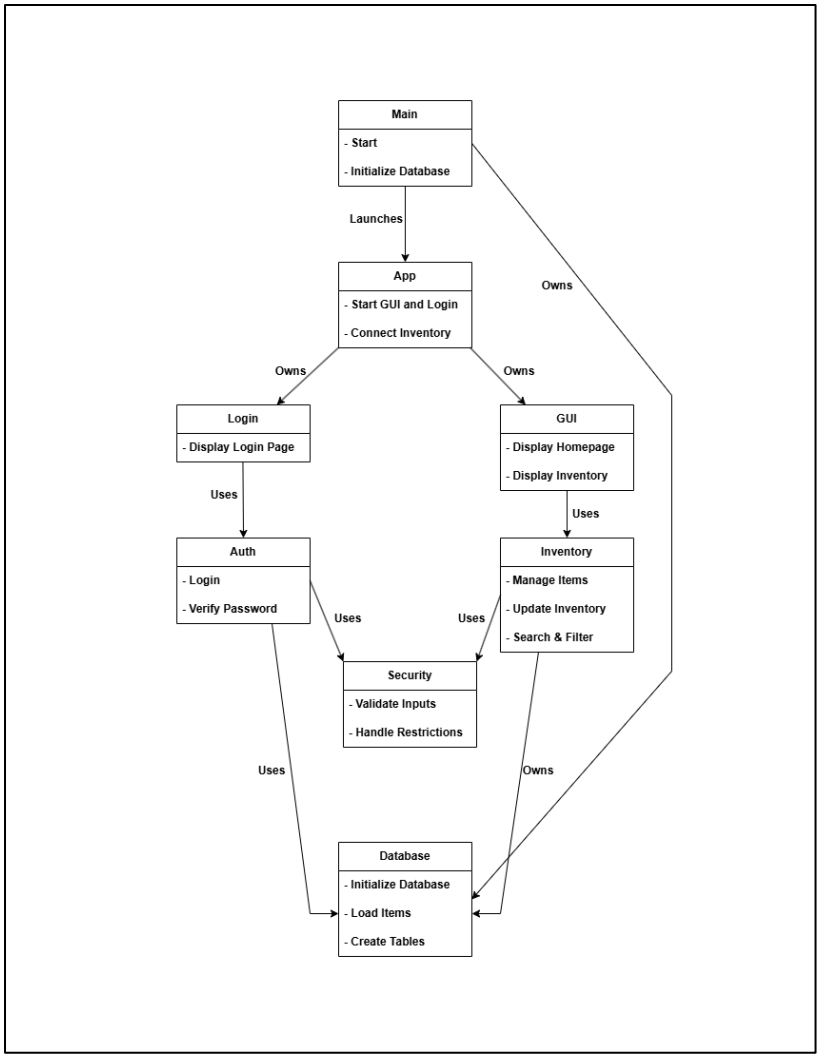
Entity Relationship Diagram

The graph above shows an Entity Relationship (ER) diagram representing the Lab Inventory Management System. The Lab Inventory Management System has three main entities: User, Lab Areas, and Item.

The User entity is an entity representing one person who is responsible for managing the system. The User is identified by a unique Username. The User is secured by three authentication attributes. The User is secured by the Password, Security Pin, and Salt Hash Password. The Salt Hash Password is a cryptographic technique used to secure the password against any attacks.

The 1 User can manage multiple Lab Areas. This is represented by the one-to-many relationship (1:N). The Lab Areas entity represents different sections in the lab. The identification of each lab area is done by the Location Number, which is the primary key for the entity.
The relationship between Lab Areas and Items is many-to-many (M:N). This means one lab area can have multiple items, and one item can exist in multiple lab areas.

The Item entity is the representation of the items in the inventory. The attributes of the Item entity are seven. These are Id (primary key), Name, Quantity, Category, Vendor, Date Added, and Expiration Date.



UML Class Diagram

The UML class diagram above illustrates how the application begins execution through the Main class, which is responsible for starting the program and initializing the database before launching the App class.

The App class serves as the central hub of the system, owning and coordinating three key components: the GUI, the Inventory, and the Auth (authentication) system.

The GUI class handles the visual side of the application, providing two displays, the login screen, and the inventory screen, acting as the interface between the user and the system. When a user interacts with the login display, it triggers the Auth class to begin the authentication process.

The Auth class manages the login flow directly, handling login, password verification, and password hashing. To validate user credentials securely, Auth relies on the Security class, which is responsible for validating the username and password inputs.

Once the user has been successfully authenticated, the Inventory class takes over. Inventory is responsible for managing all item-related operations, including adding items, removing items, searching the inventory, and adding items to a buy list. To carry out these operations, Inventory operations must access the Database class, which handles initializing the database, loading items, and managing the table structure that the rest of the system depends on.

5.3 Detail Code

Python 3 - Python version 3.12.3 contributes to the overall body of the product. It is used to implement various product functionalities:

1. The user will be able to interact with the Graphical user Interface (GUI)
2. The user will be able to query the database, add/remove items from the database, and sort the database
3. The user will be able to securely login, and reset user password
4. The user will be able to assign/unassign vendors to inventory items
5. The user will be able to create, and manage add/remove items to a buy-list

SQL Lite (python plug-in) - The product will use SQL Lite to create and manipulate the database

Source Code Repository

The latest version of the software source code is maintained in the following repository:

- Github: < https://github.com/Salvadel/inventory-manager >

Appendix A

A.1 Traceability Matrix

Table 3: Requirements Traceability Matrix

Req't ID	Use Case ID	Text of Requirement	Implementing Code (file name)	Sprint Completed	Test Case(s) ID
RQ67 RQ68 RQ69 RQ70 RQ71	UC20	The system shall present a login screen to the user, allow the user to enter a password, validate the entered password, grant access upon successful authentication, and deny access upon authentication failure	auth.py, database.py, and security.py	Sprint 5	TC_01
RQ1 RQ2 RQ3 RQ4 RQ5 RQ6 RQ7 RQ8 RQ9 RQ10 RQ11 RQ12 RQ13 RQ14 RQ15 RQ16	UC1 UC2 UC3	The system shall provide an option to create a new inventory item, allow the user to enter item details, validate item data before submission, generate a unique identifier for each item, store the item and its details in the database, confirm successful item creation to the user, provide a list of existing inventory items, allow the user to select an inventory item, display current item quantity, allow modification of item quantity, validate modified item data, update the item in the database, confirm successful item modification, confirm the selected item with the user, remove the item and its details from the database, and confirm the item deletion to the user	inventory.py, database.py, and security.py	Sprint 5	TC_02
RQ8 RQ17 RQ18 RQ19 RQ25	UC4 UC5 UC7 UC8	The system shall allow the user to select an inventory item, retrieve item details from the database, display item details such as quantity, category, and expiration date, display	inventory.py, and database.py		TC_03

RQ26 RQ27 RQ28 RQ29 RQ30 RQ31 RQ32 RQ33		a sort item option, retrieve inventory items matching the sort criteria, display matching inventory items, notify the user when no matching inventory items are found in response to a sort request., display a filter option, have a set number of options to filter by, have a graphical method for selecting a filter option, only display items that match the filter setting, and notify the user when no matching inventory items are found in response to a filter request			
RQ20 RQ21 RQ22 RQ23 RQ24	UC6	The system shall display a search input option, validate the search criteria when search input is submitted, retrieve inventory items matching the search criteria, display matching inventory items, and notify the user when no matching inventory items are found in response to a search request	inventory.py, database.py, and security.py		TC_04
RQ55 RQ56 RQ57 RQ58 RQ59 RQ60 RQ61	UC16 UC17 UC18	The system shall provide an option to add an item to the To-Buy list, store the To-Buy list entry in the database, allow the user to remove items from the To-Buy list, update the database after removal, display the updated To-Buy list after removal, retrieve To-Buy list records from the database, and display the To-Buy list to the user	inventory.py, and database.py		TC_05
RQ51 RQ52 RQ53 RQ54	UC14 UC15	The system shall allow the user to enter vendor information, store vendor information associated with the item, retrieve vendor records for a selected item, and display associated vendors	Inventory.py, database.py, and security.py		TC_06
RQ34 RQ35 RQ36 RQ37 RQ38 RQ39 RQ40 RQ41	UC9 UC10	The system shall display an option to create a category in the item management menu, display a category creation page with a textbox, “submit” button, and a “cancel” button, validate the category name in response to the user submitting a name, shall store the new category in the database in response to successful validation, confirm successful category creation to the user in response to storing the category, display an option to delete a category in the item management	Inventory.py, database.py, and security.py		TC_07

		menu, display a category deletion page with a textbox, “submit” button, and a “cancel” button, and remove the specified category from the database in response to successful validation			
RQ42 RQ43 RQ44 RQ45 RQ46 RQ47	UC11 UC12	The system shall display an option to assign a category in item details, display a category page showing all created categories, assign the specified category to an item in the database, display categories assigned to an item, allow the user to remove a category association, and update the database after category removal	Inventory.py, database.py, and security.py		TC_08
RQ48 RQ49 RQ50	UC13	The system shall allow the user to select a category, retrieve items associated with that category, and display those items in the inventory table	Inventory.py, and database.py		TC_09
RQ62 RQ63 RQ64 RQ65 RQ66	UC19	The system shall allow the user to select an option to generate a To-Buy List report, retrieve all items contained in the To-Buy List from the database, retrieve item details associated with each To-Buy List entry, create a text file containing all To-Buy List items and their details, and confirm successful creation of the text file to the user	Inventory.py, and database.py		TC_10
RQ72 RQ73 RQ74	UC21	The system shall allow an authenticated user to request a password change, verify the current password before accepting a new password, and confirm password changes to the user	Database.py, auth.py, and security.py		TC_11
RQ3 RQ11 RQ36 RQ69	UC1 UC2 UC9 UC20 UC21	The system shall validate item data before submission, validate modified item data, validate the category name in response to the user submitting a name, and validate the entered password	Inventory.py, database.py, auth.py, and security.py		TC_12

A.2 Test Cases and Results

Test ID: TC_01	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (03/30/26)**	
Purpose	This section will test UC20 (User Login). • Req. 67: The system shall present a login screen to the user • Req. 68: The system shall allow the user to enter a password • Req. 69: The system shall validate the entered password • Req. 70: The system shall grant access upon successful authentication • Req. 71: The system shall deny access upon authentication failure This test case will operate within the auth.py, and database.py files. This test will confirm the successful operations of the hash_password(), verify_password(), login_user(), and get_user() functions.		
Dependencies			
Pass/ Fail Conditions	The user must be able to successfully login to the application through keyboard inputs, and graphical display. If invalid credentials are entered the application displays an error message and denies access.		
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code.***		
Test Data:	Case 1: Username = LabTA, Password = COMLab123!, Expected result = User account confirmed and homepage displayed Case 2: Username = Wronguser, Password = COMLab123!, Expected result = System denies access and informs that user does not exist Case 3: Username = LabTA, Password = Wrongpass, Expected result = System denies access and informs that password is incorrect Case 4: Username = (blank), password = (blank) Expected result: system denies access		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	The system prompts the user to login	System displays the login page	P
3.	The user attempts to log in with an invalid username	System denies login and prompts the user credentials and invalid	P
4.	The user attempts to log in with an invalid password	System denies login and prompts the user credentials and invalid	P
5.	The user attempts to log in without entering any credentials	System denies login and prompts the user to enter credentials	P
6.	The user attempts to log in with valid credentials	Password text is obscured	P
7.	System validates username and password	No errors or system crashes	P
8.	Access is granted to the app	User is given access to the application	P
Comments			

Test ID: TC_02	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (03/30/26)**
Purpose	This section will test UC1, UC2, and UC3 (Add, modify, and remove inventory items). • Req. 1: The system shall provide an option to create a new inventory item • Req. 2: The system shall allow the user to enter item details • Req. 3: The system shall validate item data before submission • Req. 4: The system shall generate a unique identifier for each item • Req. 5: The system shall store the item and its details in the database • Req. 6: The system shall confirm successful item creation to the user • Req. 7: The system shall provide a list of existing inventory items • Req. 8: The system shall allow the user to select an inventory item • Req. 9: The system shall display current item quantity • Req. 10: The system shall allow modification of item quantity • Req. 11: The system shall validate modified item data • Req. 12: The system shall update the item in the database • Req. 13: The system shall confirm successful item modification • Req. 14: The system shall confirm the selected item with the user • Req. 15: The system shall remove the item and its details from the database • Req. 16: The system shall confirm the item deletion to the user This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the add_inventory_item(), remove_inventory_item(), modify_inventory_item(), remove_item(), add_item(), and modify_item() functions.	
Dependencies	• TC_01 • TC_02	
Pass/ Fail Conditions	The user must be able to successfully add, remove, and modify inventory items from the database. Updated values should be instant and accurate.	
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code***	
Test Data:	Case 1: User adds item to inventory Expected result = The item is appended to the database and viewable in the inventory view Case 2: User modifies the quantity of the item in inventory Expected results = The qualities of the item is modified, and viewable in the inventory view Case 3: User removes the item from inventory Expected results = The application confirms user intentions, and inventory item is removed from the database, no longer viewable in the inventory view	

Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects add item button	Pop up is displayed asking for item information	P
4.	User fills out item information	Inventory item is appended to the inventory view	P
5.	User clicks on newly added inventory item in inventory view	Pop up displays item information and ability to modify qualities	P
6.	User changes item information and qualities	The new item qualities are updated in the inventory view	P

7.	User clicks the delete button next to the inventory item	The item is deleted from the inventory database and no longer visible	P
Comments			

Test ID: TC_03	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (04/02/26)**	
Purpose	This section will test UC4, UC5, UC7, and UC8 (View sorted and filtered inventory items). • Req. 8: The system shall allow the user to select an inventory item • Req 17: The system shall retrieve item details from the database • Req 18: The system shall display inventory items in a tabular spreadsheet format • Req 19: The system shall display item details such as quantity, category, and expiration date. • Req 25: The system shall display a sort item option • Req 26: The system shall retrieve inventory items matching the sort criteria • Req 27: The system shall display matching inventory items • Req 28: The system shall notify the user when no matching inventory items are found in response to a sort request. • Req 29: The system shall display a filter option • Req 30: The system shall have a set number of options to filter by • Req 31: The system shall have a graphical method for selecting a filter option • Req 32: The system shall only display items that match the filter setting • Req 33: The system shall notify the user when no matching inventory items are found in response to a filter request. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the get_all_items(), sort_items(), filter_items(), view_inventory, sort_inventory(), and filter_inventory() functions.		
Dependencies	• TC_01		
Pass/ Fail Conditions	The system must be able to successfully display the inventory items in a sorted manner, and allow for filtering of items by inventory category. If the inventory database is empty the system will inform the user.		
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, the inventory database has items inside of it.***		
Test Data:	Case 1: User selects view inventory Expected result = The entire inventory database is displayed to the user Case 2: User selects sort items Expected results = The inventory displays the items organized by the quality selected by the user Case 3: User selects filter items Expected results = The inventory displays only the inventory items included in the filter quality		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects sort	Inventory items are sort by chosen quality	P
4.	User selects filter	Inventory items are filtered by chosen quality	P
Comments			

Test ID: TC_04	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (04/02/26)**	
Purpose	This section will test UC6 (Search inventory items). • Req 20: The system shall display a search input option • Req 21: The system shall validate the search criteria when search input is submitted • Req 22: The system shall retrieve inventory items matching the search criteria • Req 23: The system shall display matching inventory items • Req 24: The system shall notify the user when no matching inventory items are found in response to a search request. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the search_items(), and search_inventory() functions.		
Dependencies	• TC_01 • TC_02		
Pass/ Fail Conditions	The user must be able to successfully display inventory items relating to the search criteria. If no items fit the search criteria the system will inform the user.		
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, and existing items inside of the database exist***		
Test Data:	Case 1: User enters and existing item name into the search field Expected result = The named item and like items appear in the inventory view Case 2: User does not enter any text into the search field Expected results = The inventory view is unchanged and continues to display all items Case 3: User enters invalid string into the search field Expected results = The application does not accept the string and displays to the user that the string was invalid		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects search field	User is allowed to type a string into the search field	P
4.	User enters item name	Matching or similar inventory items are displayed in the inventory view	P
5.	User deleted item name and leaves search field empty	The entire unfiltered inventory database appears in the inventory view	P
6.	User enters invalid string into the search field	The user prompts the search field as invalid	P
Comments	Search functions exclusively with inventory item names. Search by ID, vendor, quantity, and other metrics is not a feature.		

Test ID: TC_05	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC16, UC17, and UC18 (Add, remove, and view items in to-buy list). • Req 55: The system shall provide an option to add an item to the To-Buy list. • Req 56: The system shall store the To-Buy list entry in the database. • Req 57: The system shall allow the user to remove items from the To-Buy list. • Req 58: The system shall update the database after removal. • Req 59: The system shall display the updated To-Buy list after removal.	

	• Req 60: The system shall retrieve To-Buy list records from the database. • Req 61: The system shall display the To-Buy list to the user. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the add_to_buy(), remove_from_to_buy(), get_to_buy_list(), add_item_to_buy_list(), view_to_buy_list(), and remove_item_from_to_buy_list() functions.
Dependencies	• TC_01 • TC_02
Pass/ Fail Conditions	The user must be able to successfully add, remove, and view items in the to-buy list. The list should display itself to the user without crashing the program, and in a legible format.
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, and existing items inside of the database exist***
Test Data:	Case 1: User adds item to-buy list Expected results = item is added to the to-buy list visible on the homepage Case 3: User removes item from to-buy list Expected result = item is removed from the to-buy list visible on the homepage Case 4: User selects to-buy list on homepage Expected results = List of item details such as quantity, price, and vendor are displayed

Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects item from inventory view	Item is selected and details appear	P
4.	User selects to-buy list from the item details	Another pop up is displayed asking for additional item details	P
5.	User enters item details and selects confirm	Item is appended to the to-buy list	P
6.	User selects view to-buy list from the homepage	The to-buy list appears with the newly appended item and details visible	P
7.	User selects remove next to the newly added items	The system pops up a message confirming deletion	P
8.	User selects confirm deletion	The item is deleted and no longer visible in the to-buy list	P
Comments			

Test ID: TC_06	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC14, and UC15 (Manage, and view vendors to inventory items). • Req 51: The system shall allow the user to enter vendor information. • Req 52: The system shall store vendor information associated with the item. • Req 53: The system shall retrieve vendor records for a selected item. • Req 54: The system shall display associated vendors. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the add_vendor(), remove_vendor(), update_vendor(), add_vendor_to_item(), remove_vendor_from_item(), and update_vendor_from_item() functions.	
Dependencies	• TC_01 • TC_02	

Pass/ Fail Conditions	The user must be able to successfully add, remove, and update vendors associated with inventory items. The vendor shall be associated with a specific item and stored in the database.		
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, and existing items inside of the database exist***		
Test Data:	Case 1: User adds vendor information to item details Expected result = Vendor is associated with the item Case 2: User removes vendor information from item details Expected results = Vendor is unassigned from the item Case 3: User changes vendor name in item details Expected results = The modified vendor name is visible and saved in the database Case 4: User enters invalid string into vendor name Expected results = A message is displayed to the user, and data not stored		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects item from inventory view	Item is selected and details appear	P
4.	User enters vendor name	Vendor name is saved to database	P
5.	User selects item from inventory view	Item details is displayed and vendor is shown	P
6.	User changes vendor name	Vendor name is modified in database	P
7.	User enters invalid string such as a buffer overflow attempt	Message displayed that name is invalid, data not stored in database	P
8.	User removes vendor name	Vendor name is removed from database	P
Comments			

Test ID: TC_07	**Designed by: Salvatore D.**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC9, and UC10 (Add, and remove categories). • Req. 34: The system shall display an option to create a category in the item management menu • Req. 35: The system shall display a category creation page with a textbox, “submit” button, and a “cancel” button • Req. 36: The system shall validate the category name in response to the user submitting a name. • Req. 37: The system shall store the new category in the database in response to successful validation. • Req. 38: The system shall confirm successful category creation to the user in response to storing the category. • Req. 39: The system shall display an option to delete a category in the item management menu • Req. 40: The system shall display a category deletion page with a textbox, “submit” button, and a “cancel” button	

	• Req. 41: The system shall remove the specified category from the database in response to successful validation. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the add_category(), delete_category(), new_category(), and remove_category() functions.
Dependencies	• TC_01
Pass/ Fail Conditions	The user must be able to successfully add, and remove categories to the database. The categories names must be valid and successfully stored in the database.
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code.***
Test Data:	Case 1: User adds new category Expected result = Message appears confirming category is saved in the database Case 2: User removes category Expected results = Message appears confirming category deletion from the database Case 3: User enters an invalid category name Expected results = Message appears informing the user the category name is invalid and changes were not saved to the database

Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	Category creation page displayed	P
4.	User enters invalid category name	Category is not saved to the database	P
5.	User enters valid category name	Category is created and saved to database	P
6.	User selects 'remove category' from inventory view	Category deletion page displayed	P
7.	User enters non-existing category name	Program prompt the category does not exist	P
8.	User enters category name	Category is deleted from the database	P
Comments			

Test ID: TC_08	**Designed by: Dohyeon Yoo**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC11 and UC12 (Assign item to category and unassign item from category). • Req. 42: The system shall display an option to assign a category in item details • Req. 43: The system shall display a category page showing all created categories • Req. 44: The system shall assign the specified category to an item in the database • Req. 45: The system shall display categories assigned to an item • Req. 46: The system shall allow the user to remove a category association. • Req. 47: The system shall update the database after category removal	

	This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the assign_category(), unassign_category(), get_categories(), and get_items_by_category() functions.
Dependencies	• TC_01 • TC_02 • TC_07
Pass/ Fail Conditions	The user must be able to successfully assign and unassign categories to items. Categories assigned must exist and be stored securely in the database. Category shall be visible in item details.
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, and categories already exist inside of the database.***
Test Data:	Case 1: User assigns an item to an existing category Expected result = Item is associated with the category; item appears when filtering by that category Case 2: User unassigns an item from a category Expected results = Item is no longer associated with the category; association removed from database Case 3: User attempts to unassign a category not assigned to the item Expected results = System displays an error message; no changes are made to the database

Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	Item is selected and item options menu is displayed	P
4.	User selects 'Assign Category' and chooses a valid category	Item is associated with the selected category and confirmation is shown	P
5.	User selects 'Assign Category' and chooses a category already assigned	System displays a message that item is already assigned to this category	P
6.	User selects 'remove category' from inventory view	System displays list of categories assigned to the item	P
7.	User selects a category to unassign	System requests confirmation before removing the association	P
8.	User confirms removal	Category association is removed from the item in the database	P
Comments			

Test ID: TC_09	**Designed by: Dohyeon Yoo**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC13 (View items by category). • Req 48: The system shall allow the user to select a category. • Req 49: The system shall retrieve items associated with that category. • Req 50: The system shall display those items in the inventory table. This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the get_items_by_category(), and get_categories() functions.	
Dependencies	• TC_01 • TC_02 • TC_07	

	• TC_08		
Pass/ Fail Conditions	The user must be able to successfully view items by category. Items visible must be assigned to that specific category and no other items outside of the category should be visible.		
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, categories exist inside of the database, and one item is assigned to a category.***		
Test Data:	Case 1: User selects a category that has items assigned to it Expected result = Only items belonging to that category are displayed in the inventory table Case 2: User selects a category with no items assigned Expected results = System informs the user that no items belong to this category Case 3: User clears the category filter Expected results = Full unfiltered inventory list is restored and displayed		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	System retrieves inventory records and displays items in tabular format	P
4.	User selects the filter dropdown menu	System displays a list of available category filter options	P
5.	User selects a category from the filter list	System retrieves items in that category and updates the inventory table	P
6.	User selects 'remove category' from inventory view	Inventory table updates to show items of the second category only	P
7.	User clears the filter selection	Full unfiltered inventory list is displayed	P
8.	User selects a category that has no items	System informs user that no items belong to this category	P
Comments	Entering category name is case sensitive 'ie: Metals != metals'		

Test ID: TC_10	**Designed by: Dohyeon Yoo**	**Executed by (on date): Salvatore D. (04/02/26)**
Purpose	This section will test UC19 (Generate To-Buy List Report). • Req 62: The system shall allow the user to select an option to generate a To-Buy List report • Req 63: The system shall retrieve all items contained in the To-Buy List from the database • Req 64: The system shall retrieve item details associated with each To-Buy List entry • Req 65: The system shall create a text file containing all To-Buy List items and their details • Req 66: The system shall confirm successful creation of the text file to the user This test case will operate within the inventory.py, and database.py files. This test will confirm the successful operations of the get_to_buy_list(), add_to_buy(), and remove_from_to_buy() functions.	
Dependencies	• TC_01 • TC_02 • TC_05	
Pass/ Fail Conditions	The user must be able to successfully generate a report of the buy list in a pdf format. The list shall contain information of all items on the to-buy list, the system will not crash	
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code, and at least one item exists in the To-Buy List.***	

Test Data:	Case 1: User generates a report with items in the To-Buy List Expected result = A text file is created containing all To-Buy List items and their details; confirmation message displayed Case 2: User attempts to generate a report with an empty To-Buy List Expected results = System informs the user the To-Buy List is empty and no file is created Case 3: User cancels report generation Expected results = No file is created and the user is returned to the To-Buy List view		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	System retrieves To-Buy List records and displays items in tabular format	P
4.	User selects 'Generate To-Buy List Report'	System retrieves all To-Buy List items and their details from the database	P
5.	System creates the text file	Text file is generated containing item names, quantities, and vendor details	P
6.	User selects 'remove category' from inventory view	Confirmation message is shown and text file is accessible	P
7.	User attempts to generate report with empty To-Buy List	System displays a message that the To-Buy List is empty	P
8.	User cancels report generation mid-process	No file is created; user is returned to To-Buy List view	P
Comments	PDF File is generated in the root of the project folder next to docs and src		

Test ID: TC_11	**Designed by: Dohyeon Yoo**	**Executed by (on date): Salvatore D. (04/04/26)**
Purpose	This section will test UC21 (Change Password). • Req 72: The system shall allow an authenticated user to request a password change • Req 73: The system shall verify the current password before accepting a new password • Req 74: The system shall confirm password changes to the user This test case will operate within the auth.py, database.py, and security.py files. This test will confirm the successful operations of the hash_password(), verify_password(), and get_user() functions.	
Dependencies	Master PIN	
Pass/ Fail Conditions	The user must be able to successfully reset the password associated with the lab username utilizing the master PIN. The password change must meet security requirements and save in the database.	
Pre-conditions:	***The system is powered on and the user the master PIN is known.***	
Test Data:	Case 1: Current password = COMLab123!, New password = NewPass456! Expected result = Password updated and confirmation message displayed to the user Case 2: Current password = WrongPass!, New password = NewPass456! Expected results = System denies the change and displays an incorrect password error Case 3: Current password = COMLab123!, New password = COMLab123! (same as current)	

	Expected results = System rejects the change and informs the user the new password must differ from the current one		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	Settings screen is displayed	P
4.	User selects 'Change Password'	System prompts the user to enter their current password or backup PIN	P
5.	User enters current password	Password text is obscured; system prompts for new password	P
6.	User selects 'remove category' from inventory view	System validates the new password and updates the database	P
7.	System displays confirmation message to the user	Password is changed and user is returned to Settings screen	P
8.	User enters incorrect current password	System denies the change and prompts the user to re-enter the correct current password	P
Comments			

Test ID: TC_12	**Designed by: Dohyeon Yoo**	**Executed by (on date): Salvatore D. (04/04/26)**
Purpose	This section will test input validation across UC1, UC2, UC9, UC20 and UC21 (Add item, modify item, create category, login, and change password). • Req. 3: The system shall validate item data before submission • Req. 11: The system shall validate modified item data • Req. 36: The system shall validate the category name in response to the user submitting a name • Req. 69: The system shall validate the entered password This test case will operate within the security.py, inventory.py, and database.py files. This test will confirm the successful operations of the validate_name(), validate_quantity(), validate_password(), and add_inventory_item() functions.	
Dependencies	• TC_01 • TC_02 • TC_07 • TC_11	
Pass/ Fail Conditions	The user must be able to successfully validate each input, the system shall not update malicious input to the database, and the system shall not crash.	
Pre-conditions:	***The system is powered on, the user has a valid password or backup security code.***	
Test Data:	Case 1: User submits an item with an empty required field Expected result = System rejects the submission and displays a validation error message Case 2: User submits an item with a negative quantity value Expected results = System rejects the value and informs the user quantity must be a positive number	

	Case 3: User enters a duplicate category name Expected results = System rejects the duplicate and informs the user the category name already exists		
Steps	**Steps to Carry out the test**	**Expected behavior to be observed**	**P/F**
1.	User loads the system	Application opens without crashing	P
2.	User logs into system	System grants access to homepage and full inventory is visible	P
3.	User selects 'add category' from inventory view	Item creation form is displayed	P
4.	User submits form with an empty required field	System displays a validation error; no data is saved to the database	P
5.	User submits form with a negative quantity	System rejects the value and displays an appropriate error message	P
6.	User selects 'remove category' from inventory view	Category creation form is displayed	P
7.	User enters a duplicate category name	System displays error that the category name already exists	p
8.	User enters valid data in all fields across all tested forms	All valid data is accepted and saved to the database correctly	P
Comments			